swotvac-saviour - Product Plan

Planning document only. No implementation in this phase.

Overview

swotvac-saviour is a self-hosted web app for exam preparation. Students upload question PDFs (supplied exercise booklets, past papers), tag each document by subject, and the app extracts individual questions, shuffles them uniformly across topics, and serves them as a game-like practice session. Shuffled interleaved practice improves retention compared to blocked practice by topic.

Core behaviours:

1. User uploads a PDF and assigns subject tags to the document.
2. The app extracts text, splits it into individual questions. Splitting is question-level only: sub-parts such as (a), (b), i., ii. and their hints stay attached to their parent question so context is preserved.
3. Questions are pooled and shuffled uniformly (seeded, reproducible).
4. Each question renders through LaTeX-compatible markup (KaTeX in the browser).
5. During play, the exact source of each question (document, page, section, exercise number) is shown in a corner badge, so the user can double-check parsing or look at the original.
6. Solutions matching is out of scope for this phase; the storage schema reserves a field for it.

Design goals: clean, minimal, modern UI; very lightweight; accessible (keyboard-first, WCAG AA contrast, semantic HTML). Single-user product: no login or auth layer. Practice sessions run until the user stops; no fixed session length.

Sample document already in repo: `exercises.pdf` (MAST30026 metric and Hilbert spaces exercise booklet) plus `learning.ipynb` experiments with PyMuPDF plain-text extraction. Observed format: "Exercise 1.1." numbering, chapter and section headings ("Metrics", "Compactness"), sub-parts (a)/(b)/i./ii., "[Hint: ...]" blocks, running page headers, unicode math in extracted text.

Architecture

Single-process self-hosted app. One FastAPI service serves both the JSON API and the static frontend. SQLite for all persistent state. PDF uploads are processed synchronously for small files; a background worker path (FastAPI BackgroundTasks) covers large documents.

```
Browser (static HTML/JS/CSS + htmx + KaTeX)
   |  JSON / form posts
   v
FastAPI app (uvicorn, one process)
```

```
   |-- web/          routes: pages, upload, tags, sessions, play
   |-- ingestion/    PDF -> structured text (PyMuPDF)
   |-- parser/       text -> Question objects (splitting, hints, sub-parts)
   |-- storage/      SQLite via sqlite3, plain SQL, no ORM
   |-- shuffle/      seeded RNG session builder
   |-- ocr/          Texo ONNX inference for hard math regions (optional module)
   |-- export/       LaTeX output (later phase)
   v
SQLite (documents, tags, questions, sessions)
+ uploads/ dir storing original PDFs (source of truth for attribution)
```

Module layout follows the existing uv `src/swotvac_saviour/` package.

Tech Stack

Python-first per user preference; nothing here justifies leaving Python.

Concern	Choice	Reason
Language	Python 3.13, uv	Already the project setup; uv manages venv and lockfile
HTTP	FastAPI + uvicorn	Typed, minimal, serves API and static files in one process
Templates	Jinja2	Server-rendered pages; no SPA build step, keeps it lightweight
Interactivity	htmx	Upload, tagging, play flow without a JS framework

Concern	Choice	Reason
Styling	Hand-written CSS (custom properties, one small file)	Full control for clean/minimal/modern; no framework weight
Database	SQLite (stdlib sqlite3)	Zero-ops, single-file, perfect for self-hosted
PDF	PyMuPDF (already installed)	Fast, gives spans with font/size/position info, page rendering for source preview
Math render	KaTeX (vendored JS)	Fast client-side LaTeX rendering; no TeX distribution needed
Math OCR	Texo + onnxruntime (optional dependency)	20M-param LaTeX OCR for hard math regions; CPU inference, self-hosted, weights pulled once
Testing	pytest	Parser and shuffle logic are pure functions, cheap to test

Deliberately avoided: Celery/Redis, Postgres, Docker-as-requirement, React/Vite build chain. Texo's browser-only variant (Texo-web) was considered and rejected:

server-side onnxruntime reuses the Python stack and keeps upload-time batching simple.

Question Extraction and Splitting

Ingestion pipeline (pure functions, CLI-testable before the web layer exists):

1. Open PDF with PyMuPDF. Get structured text per page: blocks, lines, spans with font name, size, flags. Also capture page number and any embedded TOC (doc.get_toc()). Probe on the sample: embedded TOC exists with 20 entries (chapter and section names plus page numbers), so section topics come from TOC page ranges first; font-size/weight heuristics are the fallback for PDFs without a TOC.
2. Clean noise: drop running headers/footers (repeated lines across pages, e.g. "MAST30026 MHS" and repeated section titles), page numbers, TOC pages.
3. Assign section topics: from the embedded TOC page ranges when present (sample: 20 entries), else from font size/weight heading heuristics. Sections become the topic metadata for each question ("Metrics", "Compactness").
4. Split into questions: concatenate cleaned page text first (the sample has 8 questions whose bodies cross a page boundary, so pages are joined before splitting, with repeated headers/footers stripped from mid-body positions), then a question starts at a line matching the exercise-label pattern, e.g. `Exercise 1.1.` (regex covers `Exercise X.Y.`, `Question X.Y`, `Q X.Y`), anchored to the start of a line. Sample probe found 94 pattern matches but only 91 unique labels: 3 matches are inline cross-references inside other questions' bodies, which the line-start anchor excludes from being split points; the anchor yields exactly 91 questions. Everything from that line until the next exercise label is one question body. Sub-part markers `(a)`, `i.` remain inside the body - never split points. `[Hint: ...]` blocks (14 in the sample) are captured as a separate field but stay associated with the parent question.
5. Store per question: document id, page number, section heading, exercise label, body text, hint text, extraction warnings (e.g. suspected truncation).
6. Review pass: the upload result screen lists extracted questions with counts and warnings; user can confirm or re-run with adjusted settings. Bad parses are visible immediately, not silent.

Known extraction hazards (from `output.txt`): ligatures ("de fine", "Hausdor ff"), unicode math operators, multi-line display math (cases environments), occasional mangled glyphs. Handled in the normalisation layer (see LaTeX Rendering) and flagged, not silently dropped.

Tagging and Storage

Schema (SQLite):

- `documents`: id, filename, sha256, uploaded_at, original_path
- `tags`: id, name, unique per name (subject-level, e.g. "Functional Analysis", "Algebra")
- `document_tags`: document_id, tag_id
- `questions`: id, document_id, page_number, section, label, body, hint, warnings, solution (nullable, reserved)
- `sessions`: id, created_at, seed, config (JSON: selected tags, mode), finished (nullable)
- `session_questions`: session_id, question_id, position, result (nullable: correct/retry/known)

Tags are assigned per document by the user at upload time and editable later. Questions inherit the document's tags plus the extracted section name as a finer topic. All filtering for a session is by tag and/or section.

Shuffling

- Session builder: select tags and optionally sections; no fixed question count. The deck is all matching questions, uniformly shuffled; the user plays until they stop.
- Candidate pool = all questions matching the filter, joined with their document.
- Uniform shuffle: `random.Random(seed).shuffle(pool)`. Seeded RNG means a session is reproducible; the seed is stored so a session can be re-opened or shared. A "play again with fresh shuffle" action draws a new seed from the same pool.
- Default mode is one flat uniform shuffle across all matching questions (topics interleaved at random). A stratified mode (equal take per tag before shuffle) is a config option, not the default, since the user's stated goal is uniform shuffling.
- No sub-question shuffling: a question is the atomic unit, sub-parts never separated.

LaTeX Rendering

Three tiers, applied per math region at upload time; each question stores which tier produced each of its math blocks.

1. Normalisation (Python, at extraction time): map common unicode math to LaTeX commands (e.g. math operators, Greek letters, arrows), fix ligatures, convert remaining text to LaTeX-safe markup. Inline/display math spans are detected using PyMuPDF font information (math fonts like CMMI/CMSY differ from body fonts) and wrapped for KaTeX. Sample probe confirms this signal exists: math spans use CMMI12 and MnSymbol12 against LMRoman12 body. This tier covers simple inline math and is the fast path.

2. Texo OCR (https://github.com/alephpi/Texo): for math regions that extract badly (cases environments, aligned equations, large operator layouts), crop the region from a zoomed page image using the stored span positions and run Texo, a free open-source 20M-parameter LaTeX OCR model (distilled from PPFormulaNet-S, finetuned on UniMER-1M; BLEU 0.90 on simple formulas). It runs as a plain ONNX model via onnxruntime on CPU, self-hosted with no cloud call, matching the lightweight goal; model weights are downloaded once. OCR output is LaTeX fed to KaTeX, so hard constructs render as real, selectable math instead of a static image. Run at upload time, batched per document, so play time stays fast.
3. Region image fallback: if Texo output fails KaTeX validation or confidence looks wrong, fall back to the cropped image of the original region, and flag the question for review.

The extraction review screen shows each question next to its original PDF region, so any wrong OCR is caught by the user before it enters the pool.

Export (later phase, user mentioned LaTeX delivery): generate a compilable .tex worksheet from a session. Keep the renderer pluggable so both KaTeX (play view) and pdflatex (export) consume the same normalised representation.

Game UI

One page, one question at a time. Sessions are open-ended: play continues until the user stops.

1. Setup screen: pick tags (and optionally sections), start. No count picker - the deck is the full matching pool, shuffled.
2. Play screen: current question (KaTeX-rendered), progress counter (position in deck), streak counter.
3. Actions: "Reveal" (shows hint, then solution area placeholder), "Known" / "Retry" buttons record results; "Stop" ends the session at any point; keyboard shortcuts (space, 1, 2, q) for accessible fast play.
4. Corner badge (persistent, small): source reference - document title, page, section, exercise label. Clicking opens a popover with a rendered image of the original PDF page (generated server-side by PyMuPDF), so parsing can be double-checked and the original context viewed.
5. End screen: summary - attempted, known, retry counts, per-tag breakdown, "play missed again" button.
6. Stats page: all sessions persist in SQLite; a simple dashboard shows history and per-tag accuracy over time, so the user can track retention across weeks.

Visual: minimal neutral palette, generous whitespace, system font stack, dark-mode aware via prefers-color-scheme. Accessible: semantic landmarks, visible focus, WCAG AA contrast, all actions keyboard-reachable, aria-live for question transitions.

Source Attribution

Every question row stores document_id, page_number, section, label. The corner badge and the end-screen summary both render this. The original PDF is retained in uploads/ keyed by document id; the popover renders the stored page image, so attribution is verifiable against the source file even after the extraction pipeline changes.

Phases and Milestones

1. Phase 1 - Parser core (CLI): ingestion + splitting + normalisation, runs on exercises.pdf, emits questions.json. Verified by pytest on the sample document.
2. Phase 2 - Storage and tags: SQLite schema, tag CRUD, import CLI writes questions to DB.
3. Phase 3 - Web shell: FastAPI + Jinja2 + htmx, upload with tag assignment, extraction review screen, question list per tag.
4. Phase 4 - Play mode: seeded shuffle, game UI, KaTeX rendering, corner source badge with page image popover, result recording.
5. Phase 5 - Polish and export: accessibility audit, dark mode, LaTeX worksheet export, config for stratified shuffle, Texo OCR fallback for hard math regions (optional, enable only if the review screen shows normalisation alone is not enough).

Solutions matching stays out of all phases until the user provides solution material.

Open Risks and Questions

Risks:

- Math extraction fidelity: display math (cases, aligned) from LaTeX-produced PDFs extracts imperfectly; fallback is original-region images. Likelihood high, mitigated, visible to user.
- Parser fragility across documents: other PDFs may use different numbering ("Question 5", "Part II Q7", numbered lists). Regex set is configurable per document at upload time.
- Two-column or scanned PDFs: not supported in phase 1; scanned input needs OCR (out of scope until needed).
- Hint attachment ambiguity: a hint line between two exercises must attach to the previous exercise; heuristic, verified on the sample: all 14 hint blocks close before the next exercise label, so attachment is unambiguous for this document. Re-check per new document at upload review.
- Texo OCR accuracy: BLEU 0.90 on simple expressions but 0.70 on complicated ones (SCE class), so wrong LaTeX is possible. Mitigated three ways: OCR only runs where normalisation already failed, the extraction review screen shows render next to original region before questions enter

the pool, and the region image fallback remains if KaTeX validation fails.
- Texo license is AGPL-3.0: fine for self-hosted personal use with no distribution; if the app is ever shared or hosted publicly for others, AGPL source obligations apply to the combined work. Flagged so the decision is conscious.

Decisions made (revision 2):

1. Single-user app, no login.
2. Sessions persist; stats dashboard tracks history and per-tag accuracy over time.
3. Session length is infinite - the deck is the full matching pool, user stops whenever.
4. User will supply additional sample PDFs with different question formats; parser will be validated against them in Phase 1 before the web layer is built. (Pending: the samples.)